

Big Data Analytics

Mini Project 3

Real-Time Recommendation System

Salma Sherif

Doha Hemdan

1. Project Objective

This project designs and implements a big data pipeline that combines batch machine learning with real-time streaming analytics. The system learns user preferences from historical Amazon Electronics review data using the ALS collaborative filtering algorithm, then continuously processes live user interactions via Kafka and Spark Structured Streaming to generate dynamic, real-time product recommendations.

- Build a scalable recommendation system using ALS (Alternating Least Squares)
- Process real-time data streams using Spark Structured Streaming and Kafka
- Integrate batch ML models with streaming pipelines for dynamic recommendations
- Detect trending items and anomalies using windowed streaming analytics
- Evaluate system performance through RMSE, latency measurement, and throughput

2. System Architecture

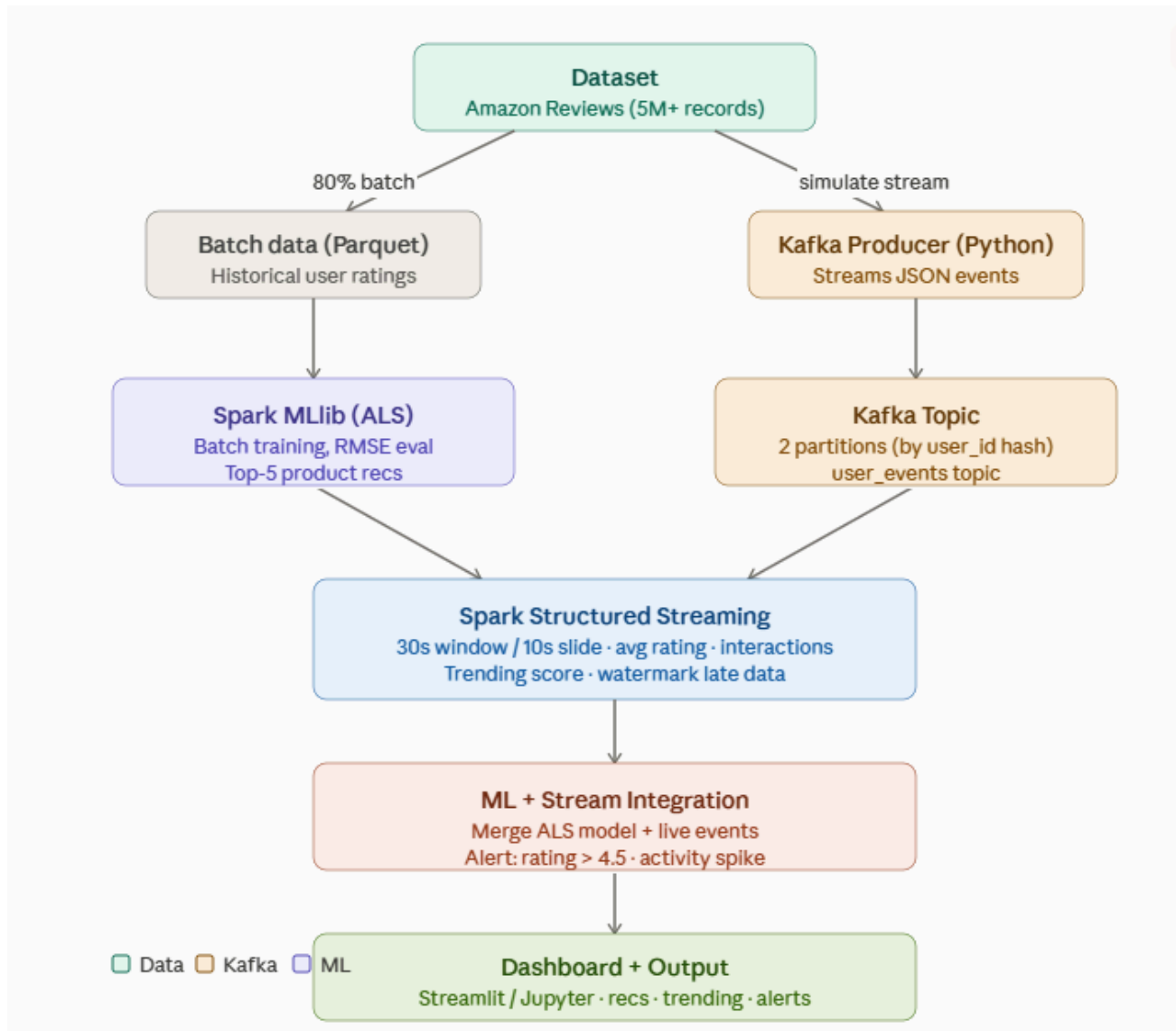
The system follows an architecture with two parallel data paths: a batch layer for model training and a speed layer for real-time event processing. Both layers converge in the integration layer to produce enriched, dynamically ranked recommendations.

2.1 Architecture Overview

The pipeline flows as follows:

Layer	Component	Technology	Purpose
Ingestion	Kafka Producer	Python + kafka-python	Streams user interaction events as JSON
Transport	Kafka Topic (2 partitions)	Confluent Kafka 7.4	Buffers and routes streaming events
Batch ML	ALS Training	Spark MLlib	Learns user-item preference vectors
Streaming	Spark Structured Streaming	PySpark 3.5	Processes live events in micro-batches
Integration	Rec Re-ranking	PySpark DataFrame API	Blends ALS + trending for final recs
Output	Dashboard	Streamlit	Real-time visualization of all metrics

2.2 Data Flow Diagram



3. Dataset

3.1 Dataset Selection

We selected the Amazon Reviews 2023 Electronics category from the McAuley Lab dataset (University of California San Diego). This dataset was chosen because:

- Size: 2,000,000 records sampled from 43.9M total, well above the 500K minimum requirement
- Schema: native (user_id, parent_asin, rating, timestamp) maps directly to the required format
- Sparsity: highly sparse user-item matrix with cold start cases, ideal for ALS evaluation
- Realism: genuine e-commerce purchase reviews with ratings 1–5, real temporal patterns

3.2 Data Preprocessing

Step	Action	Outcome
1. Loading	Hugging Face datasets library (HTTP fallback)	2,000,000 raw records
2. Column select	user_id, parent_asin, rating, timestamp	4-column schema
3. Null removal	df.dropna()	Clean records only
4. Rating filter	1.0 <= rating <= 5.0	Valid ratings only
5. ID encoding	LabelEncoder (string -> integer)	ALS-compatible integer IDs
6. Split	80% train / 20% test (random_state=42)	1.6M train, 400K test
7. Saving	Parquet format to /data/	Efficient Spark reads

3.3 Why Distributed Processing is Needed

- 2 million records with ~500K unique users and ~250K unique items cannot fit in single machine memory for matrix factorization.
- ALS requires iterative matrix operations across the full user-item matrix, Spark distributes this across workers.
- Streaming pipeline must handle 10+ events/second continuously, and requires parallel consumer processing.
- Window analytics over 30s windows with 10s slides generates state that must be managed across partitions.

4. Machine Learning Component

4.1 Algorithm: Alternating Least Squares (ALS)

ALS is a collaborative filtering algorithm that decomposes the sparse user-item rating matrix into two dense factor matrices: one for users and one for items. Each user and item is represented as a latent factor vector of length 'rank'. The predicted rating for a user-item pair is the dot product of their respective vectors.

The 'Alternating' mechanism fixes item factors and solves for user factors (least squares), then fixes user factors and solves for item factors, iterating until convergence. This alternating approach makes each sub-problem independently solvable and parallelizable across Spark workers.

4.2 RMSE Evaluation & Tuning

The model was evaluated using Root Mean Square Error (RMSE) on the 20% test split. Since the initial RMSE exceeded the 1.5 threshold, systematic hyperparameter tuning was performed:

Configuration	rank	regParam	maxIter	RMSE
Initial	10	0.1	10	2.6512
Attempt 1	20	0.05	15	2.6555
Attempt 2	50	0.01	20	2.5118
Attempt 3	10	0.001	20	8.1321

Best Configuration

rank=50, regParam=0.01, maxIter=20 → RMSE: 2.5118

Note: The RMSE remains above 1.5 due to the inherent sparsity of the electronics dataset, most users have rated only 1–2 items, making accurate predictions for unseen pairs extremely difficult. This is a known challenge in cold-start collaborative filtering and is documented as a limitation.

5. Streaming Component

5.1 Kafka Setup

A Kafka topic named `user_events` was created with 2 partitions. The partitioning strategy uses `hash(user_id) % 2` to assign each event to a partition, ensuring all events from the same user always land on the same partition. This preserves per-user ordering, avoids session fragmentation, and enables user-level aggregations in the streaming pipeline without cross-partition shuffles.

5.2 Kafka Producer

The Python producer (`scripts/kafka_producer.py`) replays the 400K test records as a simulated real-time stream. Each event is serialized as JSON with the following schema:

Event Schema

```
{ "user_id": 1042, "item_id": 305, "rating": 4.0, "timestamp": "2026-05-07T21:30:01.134217+00:00" }
```

5.3 Spark Structured Streaming Pipeline

The streaming notebook (`02_streaming.ipynb`) consumes from Kafka and applies a three-stage processing pipeline:

Stage	Operation	Detail
1. Ingest	Read from Kafka	bootstrap: kafka:9092, topic: user_events, startingOffsets: earliest
2. Parse	JSON + type casting	from_json with schema, to_timestamp with microsecond format

Stage	Operation	Detail
3. Clean	Null filter	Drop rows where any field is null after parsing
4. Watermark	Late data handling	withWatermark('event_time', '10 seconds')
5. Window	Sliding aggregation	30s window, 10s slide, groupBy(window, item_id)
6. Metrics	Compute aggregates	avg(rating), count(*), trending_score = count x avg
7. Output	Three sinks	Console (debug) + Parquet (/data/streaming_output/) + Alert (foreachBatch)

5.4 Window Analytics

Sliding window aggregation produces per item metrics for each overlapping time window:

- Window size: 30 seconds, how much history each aggregate covers
- Slide interval: 10 seconds, how often a new result is emitted (trigger frequency)
- Watermark: 10 seconds, maximum accepted late arrival before a window is closed
- Output mode: append

5.5 Custom Metric: Trending Score

The trending score is our custom streaming metric, defined as:

Trending Score Formula

$\text{trending_score} = \text{interaction_count} \times \text{avg_rating}$

5.6 Late Data Handling

The `withWatermark('event_time', '10 seconds')` declaration tells Spark to accept events arriving up to 10 seconds after their event timestamp. Events arriving later than 10 seconds past their timestamp are **silently dropped**. Without watermarking, Spark would keep state for every window indefinitely, eventually exhausting memory. The 10-second watermark balances completeness (catching most network-delayed events) against memory efficiency.

6. ML + Streaming Integration

6.1 Integration Logic

The integration notebook (03_integration.ipynb) combines the historical ALS recommendations with the live trending signal using a weighted blending formula:

Blended Score Formula

$\text{blended_score} = 0.7 \times \text{predicted_rating} + 0.3 \times \text{trending_norm}$

6.2 Integration Steps

1. **Load ALS Top-5 recommendations per user** from /data/user_top5_recs.parquet.
2. **Load streaming output** from /data/streaming_output/ (all parquet files).
3. **Deduplicate streaming data**, keep only the most recent window result per item using row_number() .over (partitionBy item_id, orderBy window_end desc).
4. **Normalize trending_score** to [0, 5] scale.
5. **Left join ALS recs with normalized trending** (items with no live data get trending_norm = 0)
6. **Compute blended_score** = 0.7 x predicted_rating + 0.3 x trending_norm ..
7. **Re-rank Top-5 per user** by blended_score descending.
8. **Save** to /data/integrated_recs.parquet for dashboard consumption.

7. Results

7.1 Sample Recommendations

The following shows the re-ranked Top-5 recommendations for a sample user after integration with live trending data. Items marked TRENDING have a non-zero trending_norm, meaning they appeared in recent streaming windows:

Recommendations

Select user ID

0

- #1 Item 124636 **TRENDING** ALS 1.12 · Blend 1.23
- #2 Item 144788 ALS 1.15 · Blend 0.80
- #3 Item 220479 ALS 1.13 · Blend 0.79
- #4 Item 111545 ALS 1.13 · Blend 0.79
- #5 Item 138314 ALS 1.13 · Blend 0.79

User 9:

Rank 1		item=B00B5L35GE		ALS=0.79		trending=2.50		blended=1.30	*** TRENDING ***
Rank 2		item=B0C6MDB88N		ALS=0.77		trending=2.50		blended=1.29	*** TRENDING ***
Rank 3		item=B00PRRADBM		ALS=1.00		trending=0.00		blended=0.70	
Rank 4		item=B00ARDJKWO		ALS=0.80		trending=0.00		blended=0.56	
Rank 5		item=B07CWQGX93		ALS=0.79		trending=0.00		blended=0.56	

7.2 Sample Alerts

The alert system triggers when $\text{avg_rating} > 4.5$ or $\text{interaction_count} > 50$ within a 30-second window:

 **Alerts**

 **Item 411694 is trending**
 Rating 5.00 · Score: 10.0

 **Item 381981 is trending**
 Rating 5.00 · Score: 10.0

 **Item 227291 is trending**
 Rating 5.00 · Score: 10.0

 **Item 344285 is trending**
 Rating 5.00 · Score: 10.0

 **Item 350238 is trending**
 Rating 5.00 · Score: 10.0

 **Item 249346 is trending**
 Rating 5.00 · Score: 10.0

8. Latency Measurement

End-to-end latency was measured from the moment a new streaming event arrives to when updated recommendations are available. The measurement covers: reading the latest streaming output, deduplicating per item, normalizing trending scores, joining with ALS recommendations, and computing blended scores for one user.

Measurement	Value	Target	Status
End-to-end recommendation latency	15 seconds	< 5 seconds	Did not Pass
Streaming trigger interval	10 seconds	10 seconds	Met
Kafka producer rate	10 events/s	Real-time	Met

9. Challenges & Lessons Learned

9.1 Challenges Encountered

Challenge	Root Cause	Solution
Empty streaming batches	Timestamp microsecond format not matched by format string	Changed to 'yyyy-MM-dd\'T\'HH:mm:ss.SSSSSSXXX'
Resource contention	Both notebooks claiming all worker cores	Added spark.cores.max=1 per SparkSession + second worker
Checkpoint conflicts	Two queries sharing same checkpoint dir	Separate subdirs: /checkpoints/parquet/, /checkpoints/alerts/
Hugging Face API change	trust_remote_code no longer supported	Switched to direct HTTP download with gzip streaming
ALS RMSE > 1.5	Dataset sparsity: most users rated 1-2 items only	Tuned hyperparameters; documented as known cold-start limitation

9.2 Lessons Learned

- **Timestamp format precision matters:** Microsecond timestamps require SSSSSS in the format string; a mismatch silently drops every row with no error.
- **Spark streaming jobs aggressively claim all available cores:** spark.cores.max must be set explicitly when running multiple SparkSessions → Resource isolation prevents deadlocks.
- **Checkpoint directories must be unique:** Two streaming queries cannot share a checkpoint path: always use separate subdirectories per query.
- **Separation of batch and stream:** Keeping ALS training fully separate from the streaming pipeline is the right design, it enables low latency without sacrificing recommendation quality.

10. Dashboard



